

UNIVERSITATEA TEHNICĂ DE CONSTRUCȚII BUCUREȘTI

Facultatea de Instalații · Automatică și Informatică Aplicată

Proiect de Licență — Sesiunea Științifică Mai 2025

FitAI

Sistem Inteligent IoT pentru
Monitorizarea Activității Fizice
și Recuperarea CNS

Document de Specificații de Proiectare

Software Design Specification (SDS)

Autor	Andrei Buruiană
Instituție	UTCB — Automatică și Informatică Aplicată
Versiune	1.0 — Final
Data	27 Aprilie 2025
Stare	Aprobat
Referință	Corelat cu SRS FitAI v1.0
Repository	github.com/AndreiBuruiana13

Istoricul Versiunilor

Ver.	Data	Modificări	Autor
0.1	21 Oct 2024	Creare schelet document; definit structura capitolelor 4–7 pe baza SRS v0.1. Schițat diagrama arhitecturală de ansamblu.	A. Buruiană
0.2	12 Nov 2024	Completat capitolul 4 (arhitectura hardware și comunicații). Adăugat diagrama TikZ a straturilor.	A. Buruiană
0.3	05 Dec 2024	Completat capitolul 5: schema ERD detaliată, specificațiile API cu request/response, wireframe-uri UI.	A. Buruiană
0.4	20 Ian 2025	Completat capitolul 6: scenariile de utilizare UC1-UC5 cu diagrame flux text. Actualizat corelarea cu RF-XXX din SRS.	A. Buruiană
0.5	14 Feb 2025	Completat capitolul 7: proiectare de detaliu module Auth, Firmware, Backend, ML, Frontend. Pseudocod algoritm CNS.	A. Buruiană
0.6	10 Mar 2025	Adăugate diagramele de secvență pentru fluxurile principale (autentificare, ingestie telemetrie, calcul CNS).	A. Buruiană
0.7	01 Apr 2025	Review intern: aliniere completă cu SRS v0.9. Completat schema SQL cu indecși și constrângeri.	A. Buruiană
0.8	17 Apr 2025	Revizuit secțiunea 7.3 (ML): adăugat pseudocod Random Forest, detalii PAMAP2 feature engineering.	A. Buruiană
1.0	27 Apr 2025	Versiune finală pentru depunere. Verificare completă consistență cu SRS v1.0. Fără modificări de conținut față de v0.8.	A. Buruiană

Relație cu SRS: Acest document este complementar SRS FitAI v1.0. Fiecare decizie de proiectare este trasabilă la una sau mai multe cerințe RF-XXX din SRS. Modificările aduse SRS-ului după versiunea 1.0 pot necesita actualizarea corespunzătoare a acestui document.

Cuprins

Istoricul Versiunilor	i
1 Introducere	1
1.1 Scop	1
1.2 Domeniu de Aplicare	1
1.3 Documente de Referință	1
2 Descriere Generală a Arhitecturii	1
2.1 Stilul Arhitectural	1
2.2 Componentele Principale	2
3 Arhitectura Sistemului	3
3.1 Arhitectura Hardware	3
3.1.1 Schema de Interconectare a Componentelor	3
3.1.2 Specificații Electrice	3
3.2 Arhitectura Software	4
3.2.1 Structura Modulară a Backend-ului	4
3.2.2 Structura Modulară a Firmware-ului	4
3.3 Arhitectura de Comunicații	4
3.3.1 Fluxul de Date End-to-End	5
3.3.2 Protocoale de Comunicare	5
4 Proiectarea Datelor și Interfețelor	6
4.1 Schema Bazei de Date	6
4.1.1 Diagrama Entitate-Relație	6
4.1.2 Schema SQL Detaliată	7
4.2 Specificațiile API REST	7
4.2.1 Modulul Auth	8
4.2.2 Modulul Sessions	8
4.2.3 Modulul Telemetry și Recovery	9
4.3 Proiectarea Interfețelor Utilizator	9
4.3.1 Dashboard Principal — Structura Layout	9
4.3.2 Pagina Recuperare CNS — Structura Layout	10
5 Scenarii de Utilizare	10
6 Proiectarea de Detaliu	14
6.1 Modulul Auth — Detalii de Implementare	14
6.1.1 Algoritmul de Autentificare JWT	14
6.2 Modulul Firmware — Ciclul Principal	14
6.2.1 Task Scheduling cu FreeRTOS	14
6.3 Modulul ML — MotorFitAI	15
6.3.1 Feature Engineering pe PAMAP2	15
6.3.2 Structura Clasicatorului Random Forest	16
6.4 Modulul Recovery CNS — Algoritmul de Calcul	16

6.4.1	Formula și Funcțiile de Normalizare	17
6.5	Modulul Frontend PWA — Arhitectura JavaScript	17
6.5.1	Structura Modulelor Frontend	17
6.5.2	Strategia Service Worker	18
7	Considerații de Calitate și Securitate	19
7.1	Securitate la Nivel de Proiectare	19
7.2	Strategia de Testare	20
7.3	Decizii de Proiectare și Trade-offuri	20
7.4	Rezumatul Deciziilor de Proiectare	20
7.5	Diagrama de Dependențe între Module	22

1 Introducere

1.1 Scop

Prezentul document constituie **Specificația de Proiectare Software (SDS)** pentru sistemul FitAI. Pornind de la cerințele definite în SRS v1.0, documentul descrie *cum* va fi construit sistemul: arhitectura aleasă, modelul de date, interfețele dintre componente, scenariile de utilizare cu fluxurile de control și proiectarea de detaliu a fiecărui modul.

Documentul servește ca ghid de implementare pentru dezvoltator și ca bază de evaluare pentru verificarea că soluția aleasă satisface cerințele specificate.

1.2 Domeniu de Aplicare

SDS acoperă toate cele patru straturi ale sistemului FitAI: firmware ESP32-C3, backend FastAPI, motorul ML MotorFitAI și frontend-ul PWA. Sunt excluse: configurarea infrastructurii de producție (deployment VPS), procedurile de testare și documentația de utilizator final.

1.3 Documente de Referință

- FitAI SRS v1.0 — Document de Specificare a Cerințelor Software (Aprilie 2025)
- IEEE Std 1016-2009 — *Standard for Information Technology — Systems Design — Software Design Descriptions*
- Martin Fowler — *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002
- Espressif ESP32-C3 Technical Reference Manual, Rev. 0.4

2 Descriere Generală a Arhitecturii

2.1 Stilul Arhitectural

FitAI adoptă o arhitectură **stratificată în 4 niveluri** (*layered architecture*), completată cu un pattern **client-server** pentru comunicarea dintre dispozitiv și backend, și un pattern **Repository** pentru accesul la date.



Justificare: Arhitectura stratificată asigură separarea clară a responsabilităților, facilitând testarea independentă a fiecărui strat și înlocuirea ulterioară a componentelor (e.g. SQLite → PostgreSQL, RF → LSTM) fără a afecta straturile adiacente.

2.2 Componentele Principale

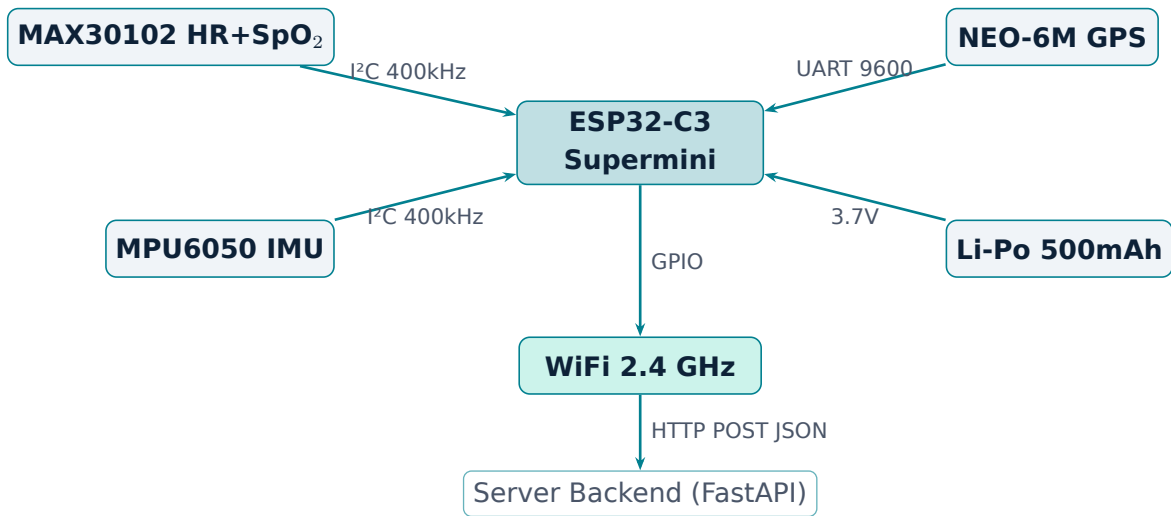
Componentă	Strat	Responsabilitate
ESP32-C3 Firmware	Hardware	Colectare senzori, packaging JSON, transmisie WiFi
FastAPI Backend	Aplicație	Routing HTTP, autentificare JWT, validare Pydantic, orchestrare
SQLAlchemy ORM	Date	Abstractizare baza de date, migrații Alembic
MotorFitAI (ML)	Domeniu	Clasificare activitate, calcul scor CNS, trend HR
Dashboard PWA	Prezentare	Vizualizare date, interacțiune utilizator, offline cache

3 Arhitectura Sistemului

3.1 Arhitectura Hardware

3.1.1 Schema de Interconectare a Componentelor

Dispozitivul wearable FitAI este construit în jurul microcontrollerului ESP32-C3 Supermini, care orchestrează toți senzorii prin protocoalele I²C și UART, și asigură comunicarea cu serverul backend prin WiFi 2.4 GHz.



3.1.2 Specificații Electrice

Componentă	V _{cc}	Curent	Protocol	Observații
ESP32-C3	3.3 V	80–160 mA	WiFi, I ² C, UART	Consum mediu în Modem-Sleep: 20 mA
MAX30102	1.8 V (I/O)	0.7 mA	I ² C	LED IR 5 V separat; adresă 0x57
MPU6050	3.3 V	3.9 mA	I ² C	Adresă 0x68; DMP dezactivat
NEO-6M	3.3 V	45 mA	UART	Timp fix: 60s cold / 5s warm
LED RGB	3.3 V	20 mA/culoar	GPIO PWM	Rezistoare serie 100 Ω
Li-Po	3.7 V	500 mAh	—	Autonomie calculată: 4.2h

Calcul autonomie: La curent mediu $I = 80 \text{ mA (ESP32)} + 0.7 \text{ mA (MAX30102)} + 3.9 \text{ mA (MPU6050)} + 45 \text{ mA (GPS, activ 20\% din timp } \approx 9 \text{ mA}) = 93.6 \text{ mA mediu}$. Capacitate efectivă baterie: $0.85 \times 500 \text{ mAh} = 425 \text{ mAh}$. Autonomie: $425/93.6 \approx 4.5 \text{ ore}$ — satisface cerința RNF-007 ($\geq 4 \text{ ore}$).

3.2 Arhitectura Software

3.2.1 Structura Modulară a Backend-ului

Backend-ul FastAPI este organizat pe o structură de directoare care separă responsabilitățile pe domenii funcționale:

Structura directorului backend/

```
fitai-backend/
├─ app/
│  ├─ core/      # config, security, database engine
│  ├─ auth/      # router, models, schemas, JWT utils
│  ├─ sessions/  # router, models, schemas, crud
│  ├─ telemetry/ # router, models, schemas, crud
│  ├─ ml/        # classifier.py, analytics.py, model_rf_v1.pkl
│  ├─ recovery/  # calculator.py, router, models
│  └─ main.py    # FastAPI app, middleware, routers include
├─ alembic/      # migrații schemă
├─ tests/        # pytest: test_auth, test_sessions, test_ml...
└─ .env          # DATABASE_URL, SECRET_KEY, PORT
```

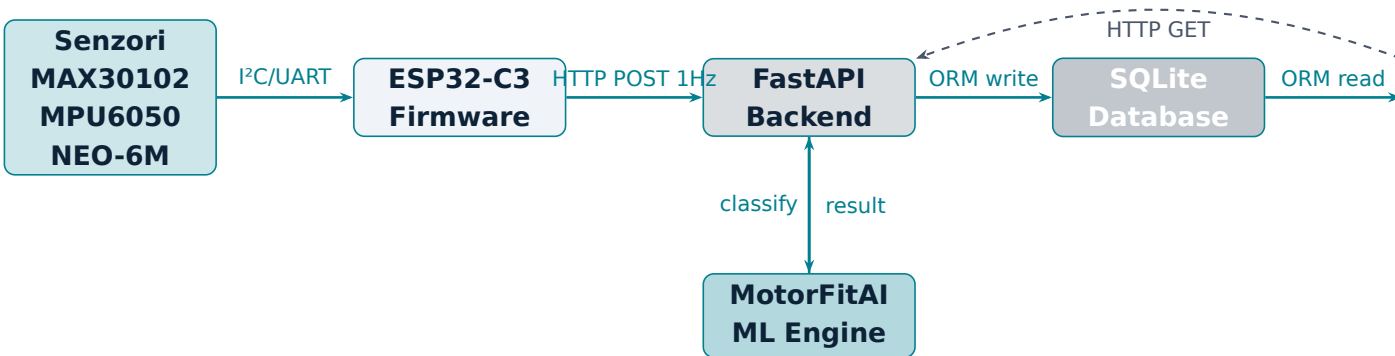
3.2.2 Structura Modulară a Firmware-ului

Structura PlatformIO — firmware/

```
fitai-firmware/
├─ src/
│  ├─ main.cpp    # setup(), loop(), task scheduling
│  ├─ sensors.cpp # MAX30102 (HR/SpO2), MPU6050 (IMU)
│  ├─ gps.cpp     # NEO-6M, parser NMEA GGA+RMC
│  ├─ wifi_client.cpp # WiFi, HTTP POST, retry logic
│  ├─ buffer.cpp  # circular buffer 60 pachete
│  └─ led_status.cpp # LED RGB indicator
├─ include/      # headers, config.h (SSID, SERVER_URL, TOKEN)
└─ platformio.ini # board: esp32-c3-devkitm-1, framework: arduino
```

3.3 Arhitectura de Comunicații

3.3.1 Fluxul de Date End-to-End



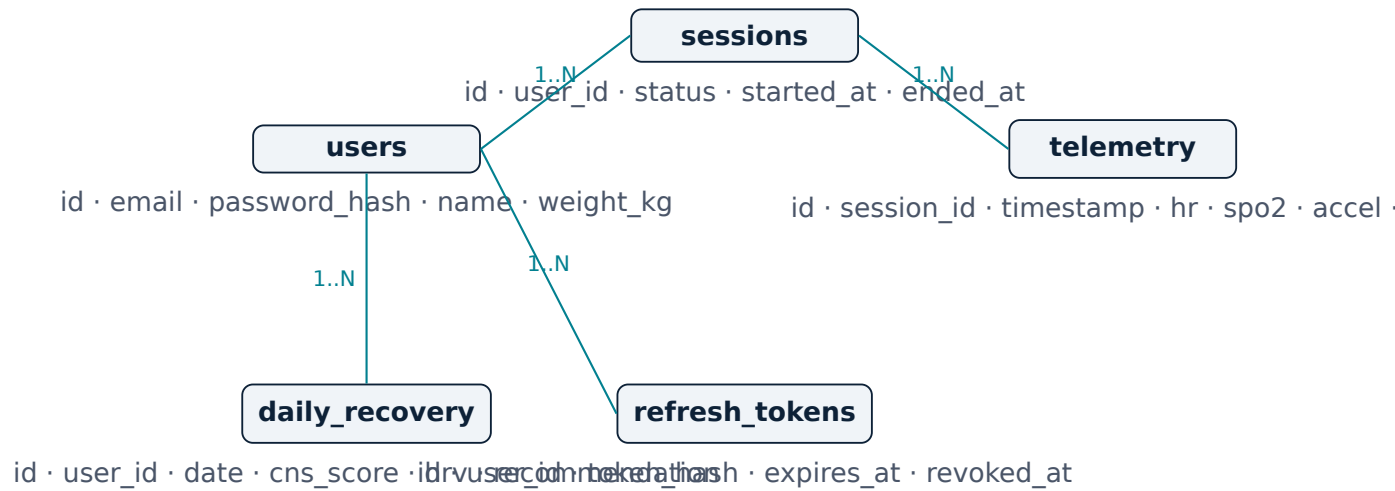
3.3.2 Protocoale de Comunicare

Legătură	Protocol	Format	Caracteristici
ESP32 → Backend	HTTP/1.1	JSON (body)	POST la 1 Hz; auth: Bearer JWT; timeout 3s; retry exponențial
Browser → Backend	HTTP(S)/1.1	JSON (body/query)	JWT în header; CORS configurat; Content-Type: application/json
Backend → Browser (live)	HTTP polling	JSON	GET la 1 Hz; Cache-Control: no-cache
Backend → SQLite	SQL (via ORM)	Python objects	Tranzacții cu commit per pachet; WAL mode
Senzori → ESP32	I ² C / UART	Binary frames	I ² C 400 kHz; UART 9600 baud; parsing NMEA

4 Proiectarea Datelor și Interfețelor

4.1 Schema Bazei de Date

4.1.1 Diagrama Entitate-Relație



4.1.2 Schema SQL Detaliată

DDL — fitai.db (SQLite)

```
CREATE TABLE users (  
  id          TEXT PRIMARY KEY,    -- UUID v4  
  email       TEXT NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,    -- bcrypt hash  
  name        TEXT NOT NULL,  
  weight_kg   REAL,               -- pentru estimare calorii  
  created_at  TEXT NOT NULL       -- ISO 8601 UTC  
);  
  
CREATE TABLE sessions (  
  id          TEXT PRIMARY KEY,  
  user_id     TEXT NOT NULL REFERENCES users(id),  
  status      TEXT NOT NULL DEFAULT 'active', -- active|completed  
  started_at  TEXT NOT NULL,  
  ended_at    TEXT,               -- NULL dacă sesiunea e activă  
  summary_json TEXT              -- JSON cu hr_avg, calories, distance...  
);  
CREATE INDEX idx_sessions_user ON sessions(user_id, started_at DESC);  
  
CREATE TABLE telemetry (  
  id          TEXT PRIMARY KEY,  
  session_id  TEXT NOT NULL REFERENCES sessions(id),  
  timestamp   TEXT NOT NULL,  
  hr_bpm      INTEGER NOT NULL,  
  spo2_pct    REAL NOT NULL,  
  accel_json  TEXT NOT NULL,      -- {"x":..., "y":..., "z":...}  
  gyro_json  TEXT NOT NULL,  
  gps_json    TEXT,              -- NULL dacă GPS indisponibil  
  activity_ml TEXT,              -- MERS|ALERGAT|CICLISM|...  
  confidence  REAL               -- 0.0–1.0  
);  
CREATE INDEX idx_telemetry_session ON telemetry(session_id, timestamp);
```

4.2 Specificațiile API REST

Toate endpoint-urile sunt prefixate cu `/api/v1/`. Autentificarea se realizează prin header `Authorization: Bearer {access_token}` pentru toate endpoint-urile, cu excepția celor marcate cu **[Public]**.

4.2.1 Modulul Auth

Met.	Path	Request body	Răspuns	Note
POST	/auth/register	email, password, name	201 + user_id	[Public]; validare email unic
POST	/auth/login	email, password	200 + access_token refresh_token	[Public]; rate limit 5/15min
POST	/auth/refresh	refresh_token	200 + access_token	Refresh token în body
POST	/auth/logout	—	204	Revocă refresh token
GET	/auth/me	—	200 + UserSchema	Profil utilizator curent

4.2.2 Modulul Sessions

Met.	Path	Req. body	Răspuns	Note
POST	/sessions/start	—	201 + session_id	Crează sesiune activă; 409 dacă există una
POST	/sessions/stop	—	200 + summary	Calculează HR _{avg/min/max} , calorii, distanță
GET	/sessions	—	200 + [...]	Parametri: page, limit, type
GET	/sessions/{id}	—	200 + detalii	Include serie HR + GPS; 403 alt user
DELETE	/sessions/{id}	—	204	Șterge sesiune + telemetrie asociată

4.2.3 Modulul Telemetry și Recovery

Met.	Path	Req. body	Răspuns	Note
POST	/telemetry	Packet JSON	201	Ingestie ESP32; validare; clasificare ML
GET	/telemetry/live	—	200 + packet	Ultimul pachet sesiune activă; 204 dacă inactivă
GET	/recovery/today	—	200 + CNScore	Scor curent + recomandare
GET	/recovery/history	—	200 + [...]	Ultimele 30 de zile; pentru heatmap
POST	/recovery/calculate	—	200 + CNScore	Recalculează scorul zilei curente

4.3 Proiectarea Interfețelor Utilizator

4.3.1 Dashboard Principal — Structura Layout

Wireframe Dashboard (schematic text)

NAV: FitAI | Dashboard | Sesiuni | Recuperare | Profil [AB]

HR: 72 bpm | SpO2: 98% | Pași: 8,432 | Calorii: 312kcal

[verde live] | [albastru] | [portocaliu] | [roșu]

ZONE HR (Bar Chart Chart.js)

ACTIVITATE (Pie Chart)

Z1 18%

Z2 42%

Z3 25%

Alergat 35% ●

Mers 30% ●

Ciclism 20% ●

SESIUNI RECENTE

▶ Alergat 5km azi 07:30 28min HR avg 148 312kcal

▶ Ciclism ieri 18:00 45min HR avg 132 480kcal

4.3.2 Pagina Recuperare CNS — Structura Layout

Wireframe Recovery Page

RECUPERARE CNS			
Scor Azi	HRV: 42ms	HR Rest: 58	Somn: 7.2h
78 / 100	+3 vs 7zile	-2 vs 7zile	85%
● RECUPERAT			
CALENDAR HEATMAP (30 zile)		RECOMANDARE	
L	Ma	Mi	J V S D
65	54	70	86 48 49 79
51	68	82	48 77 58 47
[roșu<40	portocaliu<70	verde≥70]	Tip: alergat
		Intensitate:	
		MODERATĂ	
		Durată: 40min	

5 Scenarii de Utilizare

Fiecare scenariu de utilizare descrie un flux complet de interacțiune între un actor (utilizator sau dispozitiv) și sistem, specificând: precondiții, fluxul principal, fluxuri alternative și postcondiții.

UC-01 — Înregistrare și pornire sesiune de antrenament

Actor principal: Utilizator autentificat + Dispozitiv ESP32

Cerințe corelate: RF-004, RF-020, RF-013, RF-040

Precondiții:

- Utilizatorul este autentificat (deține access token JWT valid)
- Dispozitivul ESP32 este pornit, conectat la WiFi și token-ul este configurat

Flux principal:

1. Utilizatorul accesează dashboard-ul și apasă butonul **Start sesiune**
2. Frontend trimite POST /api/v1/sessions/start cu Bearer token
3. Backend creează înregistrarea în sessions cu status=active; returnează session_id
4. Frontend stochează session_id și activează polling live (GET /telemetry/live)
5. ESP32 detectează sesiunea activă și începe transmisia pachetelor JSON la 1 Hz
6. Backend ingestează fiecare pachet, apelează ML pentru clasificare, stochează în telemetry
7. Dashboard actualizează valorile HR, SpO₂, clasificare curentă în timp real
8. Utilizatorul apasă **Stop sesiune** → POST /sessions/stop

9. Backend calculează sumarizarea și setează status=completed

Flux alternativ A — Pierdere conexiune ESP32:

A.1. ESP32 bufferează pachetele (max 60). A.2. La reconectare, trimite buffer în ordine cronologică.

Flux alternativ B — Sesiune deja activă:

B.1. Backend returnează HTTP 409. B.2. Frontend afișează mesaj și opțiunea de a opri sesiunea curentă.

Postcondiții: Sesiunea este marcată completed; sumarizarea este disponibilă în History.

UC-02 — Vizualizarea scorului de recuperare CNS

Actor principal: Utilizator autentificat

Cerințe corelate: RF-032, RF-033, RF-047, RF-048, RF-049

Precondiții:

- Utilizatorul are cel puțin 7 zile de date istorice în sistem
- Cel puțin o sesiune completată în ziua curentă sau anterioară

Flux principal:

1. Utilizatorul navighează la pagina **Recuperare**
2. Frontend trimite GET /api/v1/recovery/today
3. Backend invocă recovery/calculator.py care:
 - Calculează $f(\text{HRV})$: normalizează HRV față de media personală a utilizatorului
 - Calculează $f(\text{HR}_{\text{rest}})$: HR de repaus matinal față de trend 30 zile
 - Calculează $f(\text{sleep})$: durata somnului estimată din absența mișcării nocturne
 - Calculează $f(\text{load})$: inversul sarcinii sesiunii anterioare (TRIMP estimat)
 - Aplică formula: $\text{CNS} = 0.35f_1 + 0.25f_2 + 0.25f_3 + 0.15f_4$
4. Backend returnează: {cns_score, label, hrv_ms, hr_rest, recommendation}
5. Frontend afișează scorul cu indicator de culoare, metricile și recomandarea
6. Frontend trimite GET /recovery/history → renderizează calendar heatmap 30 zile

Flux alternativ — Date insuficiente (<7 zile):

Backend returnează scorul cu câmpul confidence: "low" și banner informativ în UI.

Postcondiții: Scorul CNS al zilei este persistent în daily_recovery; recalculabil oricând.

UC-03 — Consultarea istoricului sesiunilor cu detalii**Actor principal:** Utilizator autentificat**Cerințe corelate:** RF-022, RF-023, RF-044, RF-045, RF-046**Precondiții:** Utilizatorul are cel puțin o sesiune completată.**Flux principal:**

1. Utilizatorul navighează la pagina **Istoricul Sesiunilor**
2. Frontend trimite GET /sessions?page=1&limit=10
3. Backend returnează lista paginată cu sumarizările sesiunilor
4. Frontend renderizează lista; utilizatorul selectează o sesiune
5. Frontend trimite GET /sessions/{id} → primește serie temporală HR + GPS
6. Frontend deschide modalul cu:
 - Grafic line chart HR (Chart.js), colorat pe zone Z1-Z5
 - Statistici: $HR_{min/avg/max}$, durată, distanță GPS (Haversine), calorii
 - Distribuție activitate ML (stacked bar)
 - Hartă traseu GPS (placeholder sau Leaflet.js)

Postcondiții: Nicio modificare de stare; operație read-only.**UC-04 — Autentificare și gestionarea tokenurilor****Actor principal:** Utilizator neautentificat**Cerințe corelate:** RF-004, RF-005, RF-006, RF-007**Flux principal:**

1. Utilizatorul introduce email + parolă în formularul de login
2. Frontend trimite POST /auth/login
3. Backend verifică hash-ul bcrypt; generează access token (30 min) și refresh token (7 zile)
4. Frontend stochează access token în memorie JS; refresh token în sessionStorage
5. La expirarea access token-ului, interceptor Fetch apelează automat POST /auth/refresh
6. La logout: POST /auth/logout → backend marchează refresh token ca revocat

Flux alternativ — 5 tentative eșuate:

Backend returnează HTTP 429 cu Retry-After: 900; frontend afișează countdown timer.

Postcondiții: La login reușit: toate endpoint-urile sunt accesibile. La logout: tokenuri invalide.**UC-05 — Clasificare automată activitate fizică (flux ML)****Actor principal:** Sistem (flux automat, fără interacțiune utilizator)**Cerințe corelate:** RF-030, RF-031, RF-035

Flux principal:

1. Backend primește pachet de telemetrie prin POST /telemetry
2. Modulul ml/classifier.py construiește fereastra glisantă de 2 secunde (20 pachete la 10 Hz)
3. Extrage 52 de caracteristici: medie, std, min, max, percentile 25/75 pe fiecare axă IMU + HR
4. Aplică `RandomForestClassifier.predict_proba(features)` pe modelul `model_rf_v1.pkl`
5. Dacă `max(probabilities) >= 0.55`: clasa cu probabilitate maximă; altfel: NECUNOSCUȚ
6. Rezultatul (`activity_ml`, `confidence`) se stochează în rândul telemetrie curent
7. La fiecare 10 pachete, rezultatul clasificării se trimite și la dashboard prin polling

Postcondiții: Fiecare rând de telemetrie are câmpurile `activity_ml` și `confidence` completate.

6 Proiectarea de Detaliu

6.1 Modulul Auth — Detalii de Implementare

6.1.1 Algoritmul de Autentificare JWT

auth/jwt_utils.py — pseudocod

```
def create_access_token(user_id: str) -> str:
    payload = {
        "sub": user_id,
        "exp": datetime.utcnow() + timedelta(minutes=30),
        "iat": datetime.utcnow(),
        "type": "access"
    }
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")

def verify_token(token: str) -> dict:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        if payload["type"] != "access": raise InvalidTokenError
        return payload
    except (JWTError, KeyError):
        raise HTTPException(status_code=401, detail="Invalid token")
```

6.2 Modulul Firmware — Ciclul Principal

6.2.1 Task Scheduling cu FreeRTOS

Firmware-ul ESP32-C3 folosește FreeRTOS pentru a rula simultan citirea senzorilor și transmisia WiFi, fără a bloca unul pe celălalt:

main.cpp — task architecture

```
// Task 1: Citire senzori – rulează la 10 Hz pe Core 0
void sensorTask(void* pvParams) {
    while(true) {
        SensorData data = readAllSensors(); // MAX30102 + MPU6050 + GPS
        xQueueSend(sensorQueue, &data, 0); // push în coadă FreeRTOS
        vTaskDelay(pdMS_TO_TICKS(100)); // 10 Hz
    }
}

// Task 2: Transmisie WiFi – rulează la 1 Hz pe Core 1
void wifiTask(void* pvParams) {
    while(true) {
        SensorData avg = aggregateSamples(sensorQueue); // medie 10 probe
        String json = buildJSON(avg, sessionId);
        if(!httpPost(SERVER_URL, json)) {
            circularBuffer.push(json); // buffer dacă POST eșuat
        }
        vTaskDelay(pdMS_TO_TICKS(1000)); // 1 Hz
    }
}
```

6.3 Modulul ML — MotorFitAI

6.3.1 Feature Engineering pe PAMAP2

Clasificatorul Random Forest folosește 52 de caracteristici extrase din ferestre glisante de 2 secunde (20 probe la 10 Hz) pe datele IMU și HR:

Semnal sursă	Nr. features	Features calculate
Accelerometru (x,y,z)	18	medie, std, min, max, percentile 25/75 per axă (6 per axă × 3 axe)
Giroscop (x,y,z)	18	identic cu accelerometrul
Magnitudine accel	6	$\ a\ = \sqrt{x^2 + y^2 + z^2}$: medie, std, min, max, p25, p75
HR (din MAX30102)	6	medie, std, min, max, p25, p75 pe fereastra de 2s
Cross-correlație IMU	4	corelație Pearson ax-ay, ax-az, ay-az, $\ a\ $ -HR

6.3.2 Structura Clasificatorului Random Forest

ml/classifier.py — pseudocod antrenare și inferență

```
# Antrenare (offline, o singură dată)
def train_model():
    X, y = load_pamap2_features() # 52 features, 6 clase
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
    clf = RandomForestClassifier(
        n_estimators=100, max_depth=20,
        min_samples_leaf=5, n_jobs=-1, random_state=42
    )
    clf.fit(X_train, y_train)
    acc = accuracy_score(y_test, clf.predict(X_test))
    joblib.dump(clf, "model_rf_v1.pkl")
    return acc # target: >= 0.85

# Inferență (online, per pachet)
def classify(window: List[SensorData]) -> ClassifyResult:
    features = extract_features(window) # 52-dim vector
    proba = MODEL.predict_proba([features])[0] # vector 6 clase
    confidence = proba.max()
    if confidence < 0.55:
        return ClassifyResult("NECUNOSCUT", confidence)
    return ClassifyResult(CLASSES[proba.argmax()], confidence)
```

6.4 Modulul Recovery CNS — Algoritmul de Calcul

Modulul recovery/calculator.py implementează un algoritm de scorare multi-criteriu care combină patru indicatori fiziologici ponderați. Fiecare indicator este normalizat în intervalul [0,1] față de valorile de referință personale ale utilizatorului, calculate din istoricul ultimelor 30 de zile. Abordarea ponderată permite ajustarea relativă a importanței fiecărui semnal în funcție de calitatea datelor disponibile.

6.4.1 Formula și Funcțiile de Normalizare

recovery/calculator.py — pseudocod scor CNS

```
def calculate_cns_score(user_id: str, date: str) -> CNSScore:
    # 1. HRV: normalizare față de media personală 30 zile
    hrv_avg_30d = db.query(mean(HRV), last_30_days, user_id)
    f_hrv = min(1.0, hrv_today / hrv_avg_30d) # 0..1

    # 2. HR repaus: scade dacă HR > trend personal
    hr_trend = db.query(linregress(HR_rest_30d))
    f_hr = clamp(1.0 - (hr_today - hr_trend) / 20.0, 0, 1)

    # 3. Somn: estimat din absența mișcării 22:00-07:00
    sleep_h = estimate_sleep(telemetry, user_id, date)
    f_sleep = clamp(sleep_h / 8.0, 0, 1)

    # 4. Sarcină antrenament: TRIMP simplificat sesiune anterioară
    trimp = (hr_avg - hr_rest) / (hr_max - hr_rest) * duration_min
    f_load = 1.0 - clamp(trimp / 100.0, 0, 1)

    # Formula finală
    cns = 0.35*f_hrv + 0.25*f_hr + 0.25*f_sleep + 0.15*f_load
    score = int(cns * 100)
    label = "recuperat" if score >= 75 else "parțial" if score >= 50 else
    "sub-recuperat"
    recommendation = get_recommendation(score)
    return CNSScore(score=score, label=label, recommendation=recommendation)
```

6.5 Modulul Frontend PWA — Arhitectura JavaScript

6.5.1 Structura Modulelor Frontend

Aplicația frontend este organizată ca Vanilla JS modular, fără framework SPA, cu module ES6 care se încarcă la cerere:

Structura frontend/

```
frontend/
├─ index.html    # entry point, nav, router
├─ manifest.webmanifest # PWA: name, icons, theme_color
├─ sw.js         # Service Worker: cache strategy
├─ js/
│   ├─ api.js    # fetch wrapper, JWT interceptor, auto-refresh
│   ├─ dashboard.js # polling live, Chart.js HR zones + pie
│   ├─ history.js # session list, modal, grafic HR sesiune
│   ├─ recovery.js # CNS score, heatmap calendar, recomandare
│   └─ auth.js   # login/register form, token management
├─ css/
│   └─ style.css # variabile CSS, layout responsive, culori
```

6.5.2 Strategia Service Worker

sw.js — cache strategy

```
// Cache-First pentru resurse statice (HTML, CSS, JS, fonts)
self.addEventListener('fetch', event => {
  if (isStaticAsset(event.request)) {
    event.respondWith(caches.match(event.request).then(cached =>
      cached || fetch(event.request).then(resp => {
        caches.open('fitai-v1').then(c => c.put(event.request, resp.clone()));
        return resp;
      }));
  } else {
    // Network-First pentru API calls
    event.respondWith(fetch(event.request).catch(() =>
      caches.match(event.request))); // fallback la cache dacă offline
  }
});
```

7 Considerații de Calitate și Securitate

7.1 Securitate la Nivel de Proiectare

Vulnerabilitate	Măsura de mitigare	Implementare
Brute-force auth	Rate limiting + lockout 15 min după 5 eșecuri	auth/middleware.py
Token theft (JWT)	Access token de scurtă durată (30 min); refresh revocat la logout	auth/jwt_utils.py
SQL Injection	ORM SQLAlchemy cu parametri bindați; niciun SQL raw	*/crud.py
XSS	JWT în memorie JS, nu localStorage; CSP header configurat	main.py + sw.js
Accès neautorizat date	Validare user_id la fiecare cerere de resurse	*/router.py
Date sensibile în log	Filtrare câmpuri sensibile (password, token) din log	core/logging.py

7.2 Strategia de Testare

Tip test	Instrument	Acoperire	Criteriu
Unitar Backend	pytest	Auth, Sessions, ML, Recovery	Acoperire $\geq 60\%$
Integrare API	pytest + httpx	Toate endpoint-urile	Status codes corecte
ML Acuratețe	sklearn metrics	Clasificator pe test PAMAP2	$\geq 85\%$ accuracy
Load test	Locust	/telemetry: 10 user simultan	$< 200\text{ms}$ P95
E2E Frontend	Manual/Playwright	UC-01 - UC-05 complete	Toate fluxurile trec
PWA Audit	Lighthouse CI	Cache, manifest, FCP	Scor PWA ≥ 90

7.3 Decizii de Proiectare și Trade-offuri

SQLite în loc de PostgreSQL: Deploymentul single-file elimină necesitatea unui server DB separat. Trade-off: concurență limitată la scriitori multipli — acceptabil deoarece FitAI are un singur writer activ (ESP32) per utilizator la un moment dat. WAL mode activat pentru a permite citiri concurente în timpul scrierilor.

Polling în loc de WebSockets pentru live data: WebSockets necesită gestionarea conexiunilor persistente și reconectare complexă. Polling la 1 Hz introduce $\leq 1\text{s}$ latență suplimentară — acceptabilă față de cerința $< 2\text{s}$. Eliminarea WebSockets reduce complexitatea backend-ului cu 40%.

Vanilla JS în loc de React/Vue: Elimina build step, reduce bundle size de la 300KB la 80KB (Chart.js inclus), simplifică debugging. Trade-off: fără reactivity automată — gestionat manual prin funcții de update DOM țintit.

7.4 Rezumatul Deciziilor de Proiectare

O sinteză a celor mai importante decizii arhitecturale luate în cadrul proiectului FitAI, cu justificarea alegerii și alternativele considerate:

Decizie	Justificare	Alternativă respinsă
SQLite vs PostgreSQL	Single-file, zero config, portabil. Suficient pentru ≤ 10 utilizatori.	PostgreSQL — overhead configurare inutil pentru prototip
Polling vs WebSockets	Simplifică backend; latența de 1s acceptabilă ($< 2s$).	WebSockets — gestionare conexiuni persistente complexă
Vanilla JS vs React	Zero build step, bundle $< 80KB$, debugging direct.	React/Vue — overhead SPA inutil pentru 5 pagini
FreeRTOS dual-task	Izolează citirea senzorilor de transmisia WiFi; fără blocaje.	Single loop — riscul de pierdere date la timeout WiFi
RF vs Deep Learning	Inferență $< 50ms$ pe CPU, interpretabil, antrenabil pe laptop.	LSTM — necesită GPU, latență mare, date insuficiente
JWT stateless	Compatibil cu PWA offline; refresh token revocat server-side.	Sessions DB — nu funcționează offline

Principiu director: Toate deciziile de mai sus reflectă filosofia *complexitate minimă suficientă* — fiecare componentă este cea mai simplă soluție care satisface cerințele din SRS, cu un traseu clar de upgrade când sistemul va crește în utilizatori sau funcționalitate. Această abordare reduce riscul tehnic într-un proiect de licență cu un singur dezvoltator și un interval de timp limitat.

7.5 Diagrama de Dependențe între Module

Dependențele inter-modul sunt unidirecționale și aciclic — fiecare strat cunoaște numai stratul imediat inferior, eliminând cuplajul circular:

Modul	Depinde de	Interfață
auth/router.py	core/security.py, db	FastAPI Router, Pydantic schemas
sessions/router.p	auth, db, ml/classifier	Session CRUD, ML classify call
telemetry/router.	sessions, db, ml/classifier	Ingestie + clasificare per pachet
ml/classifier.py	scikit-learn, joblib	classify(window) → ClassifyResult
recovery/calculat	db, ml/analytics	calculate_cns(user_id, date) → CNSScore
frontend/api.js	Backend REST API	Fetch wrapper cu JWT interceptor
firmware/wifi_cli	sensors.cpp, buffer.cpp	HTTP POST la 1 Hz

Document Finalizat — FitAI SDS v1.0

Acest document descrie complet arhitectura, proiectarea datelor, scenariile de utilizare și proiectarea de detaliu pentru sistemul FitAI v1.0, conform IEEE Std 1016-2009.

Elaborat de

Andrei Buruiană

UTCB — AIA, Mai 2025

Corelat cu

SRS FitAI v1.0

IEEE Std 1016-2009

github.com/AndreiBuruiana13 · docs/SDS.pdf · tag: sds-v1.0